

Mathematical Formulations of Liquid Neural Networks

1. Liquid Time-Constant (LTC) Networks: Core Mathematics

1.1 Fundamental Differential Equation

The dynamics of an LTC network are governed by the following system of ODEs:

$$\frac{dx_i}{dt} = -\frac{1}{\tau_i} \left[x_i(t) - f_i \left(\sum_j w_{ij} x_j(t) + I_i(t) + b_i \right) \right]$$

Where:

- $x_i(t) \in \mathbb{R}$: Hidden state of neuron i at time t
- $\tau_i \in \mathbb{R}^+$: Time constant for neuron i (the "liquid" component)
- $w_{ij} \in \mathbb{R}$: Synaptic weight from neuron j to neuron i
- $I_i(t) \in \mathbb{R}$: External input to neuron i
- $b_i \in \mathbb{R}$: Bias term
- f_i : Activation function (typically sigmoid or tanh)

1.2 Conductance-Based Synaptic Model

The biological inspiration comes from conductance-based models:

$$\frac{dx}{dt} = -\frac{1}{\tau(x, I)} \cdot x + \frac{1}{\tau(x, I)} \cdot S(x, I)$$

Where the time constant itself is state-dependent:

$$\tau(x, I) = \tau_{base} \cdot \left(1 + \sigma \left(\sum_j g_j(I_j) \cdot (E_j - x) \right) \right)$$

Here:

- $g_j(I_j)$: Synaptic conductance (nonlinear function of input)
- E_j : Reversal potential for synapse j
- σ : Sigmoid function for bounded modulation

1.3 Matrix Form for Network Dynamics

For a network with N neurons, we can write:

For a network with N neurons, we can write:

$$\frac{d\mathbf{x}}{dt} = -\mathbf{D}_\tau^{-1}\mathbf{x} + \mathbf{D}_\tau^{-1}\mathbf{f}(\mathbf{W}\mathbf{x} + \mathbf{I} + \mathbf{b})$$

Where:

- $\mathbf{x} \in \mathbb{R}^N$: State vector
- $\mathbf{D}_\tau = \text{diag}(\tau_1, \tau_2, \dots, \tau_N)$: Diagonal matrix of time constants
- $\mathbf{W} \in \mathbb{R}^{N \times N}$: Weight matrix
- $\mathbf{f}$: Element-wise activation function

2. Closed-Form Continuous-Time (CfC) Networks

2.1 The Integral Challenge

Starting with the scalar LTC equation:

$$\frac{dx}{dt} = -\frac{1}{\tau}x(t) + \frac{1}{\tau}f(I(t), x(t))$$

Using the integrating factor method, the solution involves:

$$x(t) = e^{-t/\tau}x_0 + \frac{1}{\tau} \int_0^t e^{-(t-s)/\tau} f(I(s), x(s)) ds$$

The challenge: This integral has no general closed-form solution due to the nonlinearity of f and its dependence on $x(s)$.

2.2 The CfC Approximation

For the special case where we can decouple the input dependence:

$$x(t) \approx e^{-t/\tau}x_0 + A(1 - e^{-t/\tau}) + B \cdot \Phi(I, t, \tau)$$

Where Φ is the key innovation - a bounded approximation of the integral:

$$\Phi(I, t, \tau) = \frac{\tau}{t} \left[1 - e^{-t/\tau} \right] \cdot \tilde{f}(I)$$

With $\tilde{f}$ being a time-averaged approximation:

$$\tilde{f}(I) = \frac{1}{t} \int_0^t f(I(s)) ds \approx f\left(\frac{1}{t} \int_0^t I(s) ds\right)$$

$$f(I) = \frac{1}{t} \int_0^t f(I(s)) ds \approx f\left(\frac{1}{t} \int_0^t I(s) ds\right)$$

2.3 Error Bounds and Convergence

The approximation error is bounded:

$$|x_{true}(t) - x_{CfC}(t)| \leq \epsilon(t) = \mathcal{O}\left(\frac{\|f''\|_{\infty} \cdot \text{Var}(I)}{2\tau^2}\right)$$

For piecewise constant inputs with period Δt :

$$\epsilon(t) \leq C \cdot \Delta t^2 \cdot \sup_{s \in [0, t]} |f''(I(s))|$$

2.4 Multi-Layer CfC Architecture

For deep networks, we compose CfC layers:

$$\mathbf{h}^{(l+1)} = \text{CfC}^{(l)}(\mathbf{h}^{(l)}, \mathbf{W}^{(l)}, \boldsymbol{\tau}^{(l)}, t)$$

Where each layer's dynamics are:

$$\mathbf{h}_i^{(l+1)} = e^{-t/\tau_i^{(l)}} h_i^{(l)}(0) + \left(1 - e^{-t/\tau_i^{(l)}}\right) \cdot \sigma\left(\sum_j W_{ij}^{(l)} \bar{h}_j^{(l)} + b_i^{(l)}\right)$$

With $\bar{h}_j^{(l)}$ being the time-averaged activation from the previous layer.

3. Backpropagation Through Time (BPTT) for LTC/CfC

3.1 Adjoint Method for LTC

Define the adjoint variable $\lambda(t)$:

$$\frac{d\lambda}{dt} = -\frac{\partial f^T}{\partial x} \lambda - \frac{\partial L}{\partial x}$$

With boundary condition $\lambda(T) = \frac{\partial L}{\partial x(T)}$

The gradient with respect to parameters θ :

$$\frac{\partial L}{\partial \theta} = \int_0^T \lambda^T(t) \frac{\partial f}{\partial \theta} dt$$

3.2 CfC Gradient Computation

For CfC networks, gradients are explicit:

$$\frac{\partial x(t)}{\partial \tau} = \frac{t}{\tau^2} e^{-t/\tau} [x_0 - A + B \cdot G(I, t)]$$

$$\frac{\partial x(t)}{\partial W_{ij}} = \left(1 - e^{-t/\tau_i}\right) \cdot f'(\cdot) \cdot h_j$$

This eliminates the need for solving the adjoint ODE, resulting in $\mathcal{O}(1)$ memory complexity vs $\mathcal{O}(T)$ for standard BPTT.

4. Stability Analysis

4.1 Lyapunov Stability for LTC

Consider the Lyapunov function:

$$V(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x}$$

Where $\mathbf{P}$ is positive definite. The time derivative:

$$\dot{V} = \mathbf{x}^T \mathbf{P} \dot{\mathbf{x}} = -\mathbf{x}^T (\mathbf{P} \mathbf{D}_\tau^{-1} + \mathbf{D}_\tau^{-1} \mathbf{P}) \mathbf{x} + 2\mathbf{x}^T \mathbf{P} \mathbf{D}_\tau^{-1} \mathbf{f}(\mathbf{W} \mathbf{x})$$

Stability is guaranteed if:

$$\mathbf{P} \mathbf{D}_\tau^{-1} + \mathbf{D}_\tau^{-1} \mathbf{P} - 2\mathbf{P} \mathbf{D}_\tau^{-1} \mathbf{W} \succ 0$$

4.2 Contraction Theory for CfC

CfC networks exhibit contraction properties. The Jacobian:

$$\mathbf{J}(t) = e^{-t/\tau} \mathbf{I} + (1 - e^{-t/\tau}) \text{diag}(f'(\cdot)) \mathbf{W}$$

The system is contracting if:

$$\lambda_{max}(\mathbf{J}^T + \mathbf{J}) < 0$$

For small t , this approximately requires:

$$\|\mathbf{W}\|_2 < \frac{2}{\max_i f'_i}$$

5. Information-Theoretic Properties

5.1 Memory Capacity

The memory capacity of an LTC network can be quantified using the Fisher Information Matrix:

$$\mathbf{F}_{ij} = \mathbb{E} \left[\frac{\partial \log p(\mathbf{x}|\theta)}{\partial \theta_i} \frac{\partial \log p(\mathbf{x}|\theta)}{\partial \theta_j} \right]$$

For LTC networks, the effective memory horizon is:

$$T_{eff} = \int_0^\infty \frac{|\mathcal{C}(t)|}{\mathcal{C}(0)} dt$$

Where $\mathcal{C}(t)$ is the autocorrelation function of the hidden states.

5.2 Information Flow

The mutual information between input and hidden states:

$$I(X(t); I[0, t]) = H(X(t)) - H(X(t)|I[0, t])$$

For CfC networks, this can be approximated as:

$$I(X(t); I[0, t]) \approx \frac{1}{2} \log |\mathbf{I} + t \cdot \mathbf{D}_\tau^{-1} \mathbf{\Sigma}_I \mathbf{W}^T \mathbf{W}|$$

Where $\mathbf{\Sigma}_I$ is the input covariance matrix.

6. Adaptive Time Constants

6.1 Meta-Learning of Time Constants

Time constants can be learned through gradient descent:

$$\tau_i^{(t+1)} = \tau_i^{(t)} - \eta \frac{\partial L}{\partial \tau_i}$$

With the constraint $\tau_i > \tau_{min}$ enforced via projection or parameterization:

$$\tau_i = \tau_{min} + \text{softplus}(\theta_i)$$

6.2 Context-Dependent Time Constants

For adaptive behavior:

$$\tau_i(t) = \tau_{base,i} \cdot (1 + g_\tau(\mathbf{c}(t)))$$

Where $\mathbf{c}(t)$ is a context vector and g_τ is a learned modulation function.

7. Spectral Analysis

7.1 Frequency Response

The transfer function for a single LTC neuron:

$$H(s) = \frac{K}{1 + s\tau}$$

Where $s = j\omega$ for frequency domain analysis. The magnitude response:

$$|H(j\omega)| = \frac{K}{\sqrt{1 + (\omega\tau)^2}}$$

This shows low-pass filtering with cutoff frequency $f_c = \frac{1}{2\pi\tau}$.

7.2 Network Spectral Properties

For the network transfer matrix:

$$\mathbf{H}(s) = (\mathbf{I} + s\mathbf{D}_\tau - \mathbf{W})^{-1}$$

The poles of the system are given by:

$$\det(\mathbf{I} + s\mathbf{D}_\tau - \mathbf{W}) = 0$$

8. Stochastic Extensions

8.1 Noisy LTC Dynamics

Adding noise for regularization and exploration:

$$dx_i = -\frac{1}{\tau_i} [x_i - f(\cdot)] dt + \sigma_i dW_i(t)$$

Where $dW_i(t)$ is a Wiener process. The Fokker-Planck equation for the probability density:

$$\frac{\partial p}{\partial t} = \sum_i \frac{\partial}{\partial x_i} \left[\frac{1}{\tau_i} (x_i - f_i) p \right] + \frac{1}{2} \sum_i \sigma_i^2 \frac{\partial^2 p}{\partial x_i^2}$$

8.2 Variational Inference for CfC

For uncertainty quantification, we can use variational CfC:

$$q(\mathbf{x}|I) = \mathcal{N}(\boldsymbol{\mu}_{CfC}(I), \boldsymbol{\Sigma}_{CfC}(I))$$

Where both mean and covariance are outputs of the CfC network:

$$\boldsymbol{\mu}_{CfC}(I) = \text{CfC}_\mu(I, \mathbf{W}_\mu, \boldsymbol{\tau}_\mu)$$

$$\boldsymbol{\Sigma}_{CfC}(I) = \text{CfC}_\Sigma(I, \mathbf{W}_\Sigma, \boldsymbol{\tau}_\Sigma) \cdot \text{CfC}_\Sigma^T(I, \mathbf{W}_\Sigma, \boldsymbol{\tau}_\Sigma)$$

9. Computational Complexity Analysis

9.1 Time Complexity Comparison

Operation	LTC (with ODE solver)	CfC	Standard RNN
Forward Pass	$\mathcal{O}(N^2 \cdot K \cdot p)$	$\mathcal{O}(N^2 \cdot T)$	$\mathcal{O}(N^2 \cdot T)$
Backward Pass	$\mathcal{O}(N^2 \cdot K \cdot p)$	$\mathcal{O}(N^2 \cdot T)$	$\mathcal{O}(N^2 \cdot T)$
Memory	$\mathcal{O}(N \cdot K)$	$\mathcal{O}(N)$	$\mathcal{O}(N \cdot T)$

Where:

- N : Number of neurons
- K : ODE solver steps (typically $K \gg T$)
- p : Order of ODE solver
- T : Sequence length

9.2 Numerical Stability

CfC networks avoid numerical instabilities by eliminating stiff ODEs. The condition number:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| \approx \frac{\tau_{max}}{\tau_{min}}$$

For CfC, this is explicitly controlled, while for ODE-based LTC, it can grow unbounded with stiff dynamics.